



University of Science, VNU - HCM

Lab 01: Getting Started with OpenGL

Course: **Computer Grapics**

Course ID: **CS411**

Name: **Le Tien Dat**

Student ID: **23125028**

Class: **23APCS2**

Semester 2, 2025-2026

January 30, 2026.

Contents

1	Introduction	2
2	Required Libraries and Tools	2
2.1	OpenGL	2
2.2	GLFW	2
2.3	GLAD	2
2.4	Development Environment	3
3	Project Structure	3
4	Program Explanation	3
4.1	Initialization	3
4.2	Window and Context Creation	4
4.3	Loading OpenGL Functions	4
4.4	Render Loop	4
5	Compilation and Execution	5
6	Difficulties and Challenges	5
7	Conclusion	5

1 Introduction

OpenGL is a cross-platform graphics API used for rendering 2D and 3D graphics. On modern systems, OpenGL does not provide built-in utilities for window creation or input handling. Therefore, external libraries are required to create an OpenGL context and load modern OpenGL functions.

In this assignment, an OpenGL window is successfully created on Windows using **GLFW** for window and context management, and **GLAD** for loading OpenGL function pointers. The program is developed and compiled using **VS Code** and the **MinGW-w64** compiler.

2 Required Libraries and Tools

2.1 OpenGL

On Windows, OpenGL itself does not need to be installed manually. The OpenGL implementation is provided by the graphics driver. In this project, an Intel Iris Xe integrated GPU is used, which supports OpenGL 3.3 and above.

2.2 GLFW

GLFW is a lightweight library that provides:

- Window creation
- OpenGL context creation
- Keyboard and mouse input handling

GLFW is required because OpenGL alone does not handle windowing or input.

2.3 GLAD

GLAD is an OpenGL loader generator. On Windows, the system OpenGL library exposes only OpenGL 1.1 by default. GLAD dynamically loads modern OpenGL functions (e.g., OpenGL 3.3 core functions) at runtime.

2.4 Development Environment

- Operating System: Windows 10/11
- Editor: Visual Studio Code
- Compiler: MinGW-w64 (g++)
- GPU: Intel Iris Xe Graphics

3 Project Structure

The project directory is organized as follows:

```
opengl-test/  
|-- include/  
|   |-- glad/  
|   |   '-- glad.h  
|   |-- KHR/  
|   |   '-- khrplatform.h  
|   '-- GLFW/  
|       '-- glfw3.h  
|-- lib/  
|   '-- libglfw3.a  
|-- src/  
|   |-- main.cpp  
|   '-- glad.c  
'-- build/
```

This structure separates header files, source files, libraries, and build output for clarity. In the submission, I have submitted the src folder along with the release file, since the include and lib folders are identical in all settings.

4 Program Explanation

4.1 Initialization

The program begins by initializing GLFW. If GLFW fails to initialize, the program terminates immediately.

```
if (!glfwInit()) {  
    return -1;  
}
```

Then, GLFW window hints are set to request an OpenGL 3.3 Core Profile context.

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);  
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);  
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

4.2 Window and Context Creation

A window is created using GLFW, and its OpenGL context is made current.

```
GLFWwindow* window = glfwCreateWindow(800, 600, "OpenGL Window", NULL, NULL)  
    ;  
glfwMakeContextCurrent(window);
```

4.3 Loading OpenGL Functions

GLAD is initialized using GLFW's function loader. This step is essential; without it, modern OpenGL functions would be unavailable.

```
if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)) {  
    return -1;  
}
```

4.4 Render Loop

The render loop continuously clears the screen and processes events until the window is closed.

```
while (!glfwWindowShouldClose(window)) {  
    glClearColor(0.2f, 0.3f, 0.4f, 1.0f);  
    glClear(GL_COLOR_BUFFER_BIT);  
  
    glfwSwapBuffers(window);  
    glfwPollEvents();  
}
```

Finally, GLFW resources are released before program termination.

5 Compilation and Execution

The program is compiled using the following command:

```
g++ src/main.cpp src/glad.c -Iinclude -Llib -lglfw3 -lopengl32 -lgdi32 -o build/app.exe
```

The executable is then run using:

```
build/app.exe
```

Upon successful execution, an OpenGL window with a colored background is displayed.

6 Difficulties and Challenges

Several challenges were encountered during the setup and implementation process:

- **Understanding OpenGL installation on Windows:** OpenGL is not installed separately; it depends on GPU drivers, which can be confusing for beginners.
- **GLAD configuration:** Selecting the correct OpenGL version and profile (Core vs Compatibility) was important to ensure modern OpenGL usage.
- **Header include errors:** Incorrect directory structure caused compilation errors even though VS Code auto-completion appeared to work.
- **PowerShell command syntax:** Line continuation characters differ between Command Prompt and PowerShell, leading to initial build errors.

These issues were resolved by carefully organizing project files and using a single-line compilation command.

7 Conclusion

In this assignment, an OpenGL window was successfully created using GLFW and GLAD on a Windows system. The project demonstrates the correct setup of a modern OpenGL development environment and provides a foundation for further graphics programming tasks such as rendering geometric primitives and implementing shaders.